Guion de exposición de MemoryLab para una hora

Universidad Mariano Gálvez

Ingeniería en Sistemas | Sistemas Operativos 1 | Grupo 3

Edición del 21 de octubre de 2026

Esta guía reparte una exposición de 60 minutos entre los cinco integrantes. Incluye palabras para decir en voz alta, cambios de diapositiva, acciones en la aplicación y resultados que deben comprobar. El recorrido demuestra tablas de páginas, Page Fault, reemplazo FIFO, traducción de direcciones, tablas de segmentos, base y límite, asignación contigua y no contigua y conclusiones.

Reparto del tiempo

Integrante y carné	Turno	Responsabilidad
Enmer Antonio Buch Xinic · 1990-22-15514	00:00 a 12:00	Presentación, conceptos iniciales y creación del escenario de paginación
Lester Emilio Pedro Juan · 6590-23-7214	12:00 a 24:00	Tabla de páginas, Page Fault, carga de RAM y Page Hit
Deivid Alberto Guerra Carpio · 0905-24-23552	24:00 a 36:00	FIFO y traducción de direcciones con páginas presentes y ausentes
Rebeca Alizon Najarro Duarte · 3190-23-11451	36:00 a 48:00	Segmentación, tabla, base, límite y protección
Carlos Lopez Urizar · 6590-24-18604	48:00 a 60:00	Comparación, fragmentación, uso de IA, conclusiones y preguntas

Cómo usar el guion

DECIR es el texto que pueden pronunciar tal cual. **HACER** son instrucciones para quien controla la computadora; no se leen al público. **COMPROBAR** indica el resultado esperado. **PAUSA** significa dejar que el público vea o responda; no repetir clics para llenar ese tiempo.

Cada turno contiene cuatro tramos de tres minutos. Los tiempos incluyen explicación, lectura de tablas, clics y pausas. Ensayen con cronómetro: el documento no garantiza una hora por el solo hecho de leerlo. El objetivo es una voz tranquila, aproximadamente 110 a 125 palabras por minuto durante el texto hablado, con tiempo adicional para las acciones. Las preguntas finales ya están dentro de la hora.

Cada integrante habla durante su turno y puede manejar la aplicación. Si resulta difícil escribir y explicar al mismo tiempo, un compañero puede ejecutar los pasos que narra el expositor. Una sola computadora realiza los accesos; los demás pueden señalar la lámina, vigilar el tiempo o seguir su copia del guion. Las acciones de otros usuarios sobre el mismo escenario cambiarían los resultados.

Preparación antes de que empiece el reloj

1. Abrir **http://localhost/sistemg3/public/** y comprobar que carguen el logo, el menú y los estilos.
2. Iniciar sesión con la cuenta **Administrador** que ya utiliza el grupo. Usar su contraseña vigente. Se necesita ese rol para crear escenarios y mostrar todos los formularios.
3. Abrir la presentación existente: **output/presentations/memorylab-presentacion.pptx**. Tener también esta guía disponible. Usar **Alt + Tab** para pasar entre la presentación y el navegador; comprobarlo antes de iniciar.
4. Decidir cuál será el nombre nuevo de los escenarios. En este documento usaremos **Exposición G3 Paginación 01** y **Exposición G3 Segmentación 01**. Para otro ensayo, cambiar 01 por 02 y crear escenarios nuevos; anotar sus números **#ID**.
5. Empezar con la portada de la presentación. Dejar la sesión de la app abierta, sin solicitar páginas del escenario que van a exponer.
6. Tener una hoja para escribir estos cálculos: **4 ÷ 1 = 4 marcos**, **3500 = 3 × 1024 + 428**, **2 × 1024 + 428 = 2476**, **1 × 1024 + 100 = 1124**, **8192 - 7500 = 692**.
7. Acordar las señales de tiempo: a los 10 minutos de cada turno, avisar discretamente que quedan dos; al minuto 12, pasar al siguiente integrante. En el último turno, avisar a Carlos cuando sean las 57:00.

Recorrido manual. Crear los escenarios como indica esta guía. La opción **Demostración** y el seeder preparan otros estados iniciales; mezclarlos con este recorrido cambiaría los marcos y contadores. No hace falta ejecutar archivos, comandos de base de datos ni seeders para exponer.

Pantallas que vamos a utilizar

Todas estas rutas se agregan a **http://localhost/sistemg3/public**. Se pueden abrir desde el menú lateral.

Opción del menú	Ruta
Inicio	/dashboard
Configuración de memoria	/memoria/configuracion
Procesos	/procesos
Paginación	/paginacion
Traducción de direcciones	/traduccion
Segmentación	/segmentacion
Comparación de asignación	/comparacion
Entregables académicos	/entregables

Regla para cambiar de pantalla. Comprobar siempre el escenario por nombre y **#ID**. Volver a seleccionar el proceso cuando el formulario lo pida. La selección del escenario puede conservarse en la sesión; el proceso no debe darse por seleccionado.

Dónde ver los Faults. Paginación y Traducción no tienen un indicador del total acumulado. Durante esos pasos llevar el conteo en la hoja rápida. Para comprobarlo en la aplicación, abrir **Inicio** y mostrar la tarjeta **Page Faults** del mismo escenario. El guion indica dos momentos para hacerlo.

Uso de las doce diapositivas existentes

La presentación acompaña la explicación; el navegador demuestra los resultados. No es necesario mantener una lámina en pantalla mientras se llenan formularios.

Diapositiva	Tema y momento de uso
1 · MemoryLab	Enmer abre la exposición en 00:00
2 · Equipo de trabajo	Enmer presenta a los cinco integrantes
3 · Objetivos del simulador	Enmer explica qué se demostrará
4 · Arquitectura de la aplicación	Enmer explica pantalla, servidor y persistencia brevemente
5 · Modelo persistente de memoria	Enmer explica que tablas, mapa y eventos comparten un estado guardado
6 · Paginación páginas y marcos	Lester introduce página y marco; después muestra la app
7 · PAGE_HIT PAGE_FAULT y FIFO	Lester explica Hit y Fault; Deivid retoma FIFO
8 · Traducción de dirección lógica	Deivid presenta la fórmula y después usa la tabla actual de la app
9 · Segmentación base y límite	Rebeca introduce Código y sus límites
10 · Comparación con la misma RAM	Carlos presenta el problema de los huecos
11 · Roles y verificación del proyecto	Carlos explica permisos y evidencia de pruebas
12 · Conclusiones y demostración	Carlos cierra y abre preguntas

Cifras de las láminas. La 6 ilustra RAM de 16 KB; en vivo elegimos 4 KB para llenarla rápidamente. La 8 usa la tabla ilustrativa P1 a marco 5: 1124 se traduce a 5220. En nuestro escenario, P3 estará en marco 2 y 3500 se traducirá a 2476. La 10 muestra una solicitud de 7168; en vivo compararemos 8192 y 7500. Son ejemplos distintos de las mismas reglas. El guion incluye cómo decirlo.

Pruebas históricas. Las 655 pruebas, una omisión y 4575 aserciones de la diapositiva 11 son el resultado documentado del 8 de octubre. Presentarlo como evidencia de esa revisión, sin decir que se ejecutó durante la exposición ni que es un conteo actualizado hoy.

Datos que deben permanecer iguales

Paginación: RAM **4 KB**, página **1 KB**, secundaria **16 KB**, Chrome **6 KB**. Orden de cargas **P0, P1, P3, P2**; después **Hit P0**, **P4** y **P0**. Son siete solicitudes: seis Fault y un Hit.

Segmentación: RAM **16 KB**, Editor **4 KB**; Código **1000/1200**, Datos **4000/800**, Stack **7000/600**, Heap **9000/1000**, donde cada pareja significa base/tamaño en bytes.

Enmer de 00 a 03 minutos

Presentar al grupo y el objetivo

HACER. Mostrar diapositivas 1, 2 y 3. En la 2, cada integrante puede saludar y decir su nombre; la presentación de todos debe durar aproximadamente 40 segundos en total. Mantener los carnés visibles, sin leerlos lentamente uno por uno.

DECIR.

Buenos días, licenciado y compañeros. Somos el Grupo 3 de Sistemas Operativos 1, de la carrera de Ingeniería en Sistemas de la Universidad Mariano Gálvez. Mi nombre es Enmer Antonio Buch Xinic. Hoy presentamos MemoryLab, un simulador interactivo de administración de memoria.

Nuestro equipo también está formado por Lester Emilio Pedro Juan, Deivid Alberto Guerra Carpio, Rebeca Alizon Najarro Duarte y Carlos Lopez Urizar. Todos participaremos en la explicación y en las demostraciones. En esta diapositiva pueden ver nuestros nombres y carnés.

Durante esta hora vamos a seguir lo que sucede cuando un proceso necesita consultar una parte de su memoria. Primero veremos cómo una tabla relaciona páginas con espacios de RAM. Después provocaremos un fallo de página y llenaremos la memoria para observar qué hace el algoritmo FIFO. Luego localizaremos un byte mediante una dirección lógica y mostraremos cómo un segmento protege sus límites.

Para terminar, compararemos la asignación contigua, la paginación y la segmentación. Nos interesa comprobar por qué una solicitud puede fallar aunque la suma de los espacios libres parezca suficiente.

La memoria de los diagramas es simulada. Cuando indiquemos cuatro kilobytes de RAM, estaremos configurando el modelo educativo; no estaremos reduciendo ni reservando la RAM física de esta computadora. Elegimos cantidades pequeñas para poder revisar cada operación sin perder de vista las tablas.

Al terminar esperamos poder responder tres preguntas: dónde está una página, qué hacemos si no está en RAM y qué direcciones tiene permitido utilizar un proceso.

PAUSA. Permitir el saludo de los compañeros y señalar los tres objetivos de la lámina. No iniciar sesión ni crear cuentas frente al público: la sesión se preparó antes.

COMPROBAR. La diapositiva 2 identifica a los cinco integrantes y la 3 permite reconocer el alcance académico.

Enmer de 03 a 06 minutos

Explicar proceso RAM disco página y marco

HACER. Mantener la diapositiva 3 o usar una hoja. Escribir **proceso → página → tabla → marco de RAM**. Debajo escribir **1 KB = 1024 bytes** y **4 KB ÷ 1 KB = 4 marcos**.

DECIR.

Antes de abrir la aplicación voy a aclarar las palabras que utilizaremos. Un proceso es un programa en ejecución. Para trabajar necesita instrucciones y datos, y estos ocupan posiciones de memoria. Una dirección identifica una de esas posiciones.

La RAM es la memoria principal donde el procesador puede trabajar con lo que necesita. El almacenamiento secundario nos permite mantener páginas que en este momento no están alojadas en RAM. En el dibujo lo llamaremos disco, pero sigue siendo parte del modelo simulado.

En paginación dividimos la memoria del proceso en partes del mismo tamaño llamadas páginas. También dividimos la RAM en espacios del mismo tamaño llamados marcos. La página pertenece al proceso; el marco es el lugar de la RAM que puede recibirla. Tener el mismo tamaño no significa que sean el mismo concepto.

Pensemos en piezas numeradas y casillas disponibles. La pieza número tres puede colocarse en la casilla número dos. Para recordar esa relación necesitamos una tabla. En la simulación veremos precisamente una página tres ubicada en un marco dos.

Los números comienzan en cero. Cuatro marcos se identifican como cero, uno, dos y tres. También una página de mil veinticuatro bytes tiene posiciones desde cero hasta mil veintitrés.

En esta aplicación un kilobyte equivale a mil veinticuatro bytes. Si configuramos cuatro kilobytes de RAM y páginas de un kilobyte, obtenemos cuatro marcos. Si el proceso necesita seis kilobytes, tendrá seis páginas. Solo cuatro podrán estar en RAM al mismo tiempo en nuestro ejemplo, y las otras permanecerán fuera de ella.

Esto nos permitirá observar el traslado y el reemplazo de páginas con números que podemos comprobar a simple vista.

PAUSA. Preguntar: “Si tenemos cuatro casillas y seis piezas, ¿pueden estar todas dentro al mismo tiempo?”. Esperar una respuesta y confirmar: “No; necesitamos decidir cuáles mantener en RAM”.

Enmer de 06 a 09 minutos

Presentar la aplicación y sus reglas

HACER. Mostrar brevemente diapositivas 4 y 5; después pasar al navegador y abrir **Inicio**. Señalar el menú de **Configuración de memoria**, **Procesos**, **Paginación** y **Segmentación**.

DECIR.

MemoryLab es una aplicación web. La pantalla está construida con Blade, Bootstrap y la plantilla Sneat. Livewire comunica las acciones de la pantalla con Laravel. Los servicios del servidor comprueban las reglas y la base de datos conserva los escenarios, procesos y resultados.

Lo importante de esta organización es que el mapa y la tabla representan el mismo estado. Cuando carguemos una página, la asignación se guarda; la animación nos ayuda a explicar lo ocurrido. El resultado no depende de cambiar un color en el navegador.

La aplicación maneja tres roles. Administrador puede preparar la configuración, Operador puede ejecutar actividades permitidas y Observador puede consultar resultados. Hoy usaremos una cuenta administradora para mostrar el recorrido completo con una sola computadora.

En el panel veremos indicadores de RAM total, utilizada y disponible. Si no hay un escenario seleccionado, pueden aparecer guiones. Eso significa que todavía no se está mostrando un conjunto concreto de datos. No debemos interpretar un guion como una cantidad de memoria.

Un escenario reúne su propia configuración y sus procesos. Por eso vamos a crear uno nuevo para esta exposición. Así sabemos que comenzamos con cero páginas cargadas y que los contadores pertenecen a lo que hagamos aquí.

Al cambiar de pantalla comprobaremos el nombre y el número del escenario. También elegiremos el proceso cuando corresponda. Esto evita consultar una tabla de otro ejemplo y atribuirle resultados que no le pertenecen.

Ahora vamos a preparar la memoria de cuatro marcos que acabamos de calcular.

HACER. Abrir **Configuración de memoria**. En **Escenario**, seleccionar **Crear un escenario nuevo**. Dejar el formulario listo para el siguiente tramo.

COMPROBAR. Los campos de nombre y capacidades están habilitados. Si no lo están, revisar la cuenta antes de continuar.

Enmer de 09 a 12 minutos

Crear el escenario y el proceso

HACER. En **Configuración de memoria**, llenar:

Campo	Escribir
Nombre del escenario	Exposición G3 Paginación 01
RAM total (KB)	4
Tamaño de página (KB)	1
Almacenamiento secundario (KB)	16

HACER. Comprobar **Número de marcos: 4** y pulsar **Guardar configuración** una vez. Anotar el **#ID**. Abrir **Procesos**, comprobar ese escenario y, en **Crear proceso**, ingresar **Nombre: Chrome**, **Tamaño (KB): 6**. Comprobar **Páginas requeridas: 6** y pulsar **Crear proceso** una vez.

DECIR.

Estamos creando un escenario llamado Exposición G3 Paginación 01. Le damos cuatro kilobytes de RAM y un kilobyte por página. El número de marcos se calcula automáticamente y debe indicar cuatro. El almacenamiento secundario tendrá capacidad para dieciséis kilobytes.

Guardamos y anotamos el identificador. En la pantalla Procesos elegimos ese mismo escenario. Ahora creamos Chrome con un tamaño de seis kilobytes. Chrome es el nombre de nuestro proceso de ejemplo; escribirlo aquí no abre otra aplicación real en la computadora.

Como seis dividido entre uno es seis, aparecen seis páginas requeridas. Sus números serán P0, P1, P2, P3, P4 y P5. Crear el proceso todavía no carga ninguna de ellas en RAM. En este modelo empiezan sin marco, en almacenamiento secundario.

Por eso, al revisar la memoria tendremos cuatro marcos libres y seis kilobytes utilizados en secundaria. No hay contradicción en que el proceso sea mayor que nuestra RAM: no exigiremos mantener sus seis páginas residentes al mismo tiempo.

Hemos preparado las condiciones iniciales. Lester va a mostrar cómo consultamos la tabla y cómo cambia cuando la CPU solicita una página que todavía está fuera de RAM.

HACER. Abrir **Paginación**, comprobar **Seleccionar escenario** y elegir **Chrome** en **Seleccionar proceso**. Dejar visible la tabla.

COMPROBAR. Seis páginas en **DISCO**, ninguna con marco. RAM utilizada **0 KB**; secundaria utilizada **6 KB**. Entregar el control a Lester a las **12:00**.

Lester de 12 a 15 minutos

Leer la tabla de páginas

HACER. Mostrar diapositiva 6 y regresar a **Paginación**. Señalar las columnas **Página**, **Marco**, **Presente** y **Estado**; después señalar los cuatro marcos del mapa.

DECIR.

Mi nombre es Lester Emilio Pedro Juan. Vamos a utilizar la tabla de páginas para conectar el proceso con la memoria RAM. La lámina presenta un ejemplo de dieciséis marcos; en vivo usamos cuatro para poder llenarlos durante la demostración.

La primera columna identifica la página dentro de Chrome. La columna Marco indica su ubicación actual en RAM, si tiene una. Presente nos dice si está cargada y Estado muestra RAM o DISCO.

En este momento todas las páginas tienen un guion en Marco, dicen No en Presente y aparecen en DISCO. El guion no es el número cero. Significa que no hay un marco asignado. El marco cero sí existe y lo veremos cuando se cargue la primera página.

En el mapa aparecen cuatro espacios libres: marco cero, uno, dos y tres. Este mapa representa la RAM del escenario completo. Nuestra tabla corresponde al proceso Chrome que hemos seleccionado. Si hubiera otros procesos, sus páginas también podrían aparecer ocupando marcos en ese mapa.

Es importante aclarar que estas páginas no son sitios de Internet ni pestañas de Chrome. Son divisiones del espacio de memoria del proceso. El número de páginas depende del tamaño del proceso y del tamaño que configuramos para cada página.

Cuando la CPU solicite una página, consultaremos esta tabla. Si está presente, podremos utilizar su marco. Si está ausente, necesitaremos cargarla antes de completar el acceso.

La tabla nos permite cambiar la ubicación de una página sin cambiar el número con el que el proceso la identifica. Por eso no basta con conocer el número de página: también necesitamos la relación actual con su marco.

PAUSA. Señalar P0 y preguntar: “¿Ya tiene marco?”. Esperar y confirmar: “Todavía no; vamos a solicitarla”.

HACER. Bajar a **Solicitud de CPU** y seleccionar **Modo de simulación: Paso a paso** y **Página 0 · DISCO**. No pulsar todavía otra página.

Lester de 15 a 18 minutos

Mostrar el fallo de página paso a paso

HACER. Pulsar **Acceder a página**. Se abre el paso 1. Pulsar **Siguiente** una vez después de explicar cada estado hasta llegar al 8; son siete clics en Siguiente. Dejar entre tres y cinco segundos para observar cada paso.

DECIR.

Vamos a seguir una solicitud completa. La CPU necesita un dato de la página cero de Chrome. Por ahora la página no está en RAM y no le hemos asignado un marco.

En el paso uno se identifica la solicitud: qué proceso y qué página queremos consultar. En el paso dos se revisa la tabla de páginas. Allí encontramos que P0 no está presente.

En el paso tres comprobamos que la página sigue en almacenamiento secundario. En el paso cuatro identificamos el Page Fault o fallo de página. El término fallo no significa que hayamos escrito mal la contraseña ni que se haya arruinado el programa. Significa que la página que necesitamos todavía debe traerse a RAM.

En el paso cinco buscamos un marco disponible. Tenemos cuatro libres, así que elegimos el primero: el marco cero. Aquí todavía no necesitamos reemplazar ninguna página.

Ahora avanzamos al paso seis. En este punto la aplicación confirma la carga: P0 obtiene el marco cero. Podemos comprobar el cambio en el mapa y en la tabla.

El paso siete explica la actualización de esa tabla: marco cero, presente Sí y estado RAM. En el paso ocho se completa el acceso solicitado. La página ya está donde la CPU necesita encontrarla.

Los últimos pasos explican el resultado que ya se confirmó. No están creando copias adicionales. Podemos observar un solo marco ocupado y una sola página cargada. Así relacionamos la secuencia del acceso con un cambio concreto y verificable en memoria.

COMPROBAR. Resultado **PAGE_FAULT**, marco **0**, acceso completado. RAM utilizada **1 KB**, marcos libres **3**, secundaria utilizada **5 KB**, Faults acumulados **1**.

HACER. Al llegar al paso 8, cambiar **Modo de simulación** a **Automático**. No cambiar de modo en mitad del recorrido.

Lester de 18 a 21 minutos

Llenar los cuatro marcos en un orden controlado

HACER. En Automático, realizar estas tres solicitudes, una por una, esperando a que termine cada respuesta: **Página 1 · DISCO**, **Página 3 · DISCO**, **Página 2 · DISCO**. Pulsar **Acceder a página** una vez en cada caso. Respetar ese orden.

DECIR.

Vamos a cargar otras páginas para llenar la RAM. Primero solicitamos P1. Como estaba en disco, ocurre otro fallo y se utiliza el marco uno. Tenemos dos páginas cargadas y dos espacios libres.

Después solicitamos P3. El siguiente espacio disponible es el marco dos. Aquí vemos una relación importante: página tres y marco dos son números distintos. La página sigue siendo P3 dentro de Chrome, aunque ahora ocupe el marco dos de la RAM.

Por último solicitamos P2. Se carga en el marco tres y con eso ocupamos los cuatro marcos. Elegimos este orden a propósito para demostrar que la numeración de páginas no obliga a utilizar el mismo número de marco.

Revisemos el mapa completo: marco cero contiene P0; marco uno contiene P1; marco dos contiene P3; y marco tres contiene P2. La tabla debe mostrar esas mismas relaciones. Si señalamos P3, su fila debe indicar marco dos y presencia Sí.

La RAM utilizada ahora es de cuatro kilobytes y no quedan marcos libres. En secundaria permanecen P4 y P5, por lo que allí se utilizan dos kilobytes. El proceso sigue teniendo sus seis páginas; únicamente cambió la ubicación de cuatro de ellas.

Cada una de estas primeras cuatro solicitudes encontró una página ausente. Por eso el total acumulado es de cuatro Page Faults. RAM llena no significa que el proceso terminó ni que las páginas restantes se borraron. Significa que, para traer otra página, necesitaremos decidir cuál de las residentes saldrá.

Página cargada	Marco	Faults acumulados
P0	0	1
P1	1	2
P3	2	3
P2	3	4

PAUSA. Mostrar la tabla y el mapa juntos. Dar tiempo para que el público encuentre **P3 → marco 2**.

Lester de 21 a 24 minutos

Mostrar un Page Hit y preparar FIFO

HACER. Con la RAM llena, elegir **Página 0 · RAM** y pulsar **Acceder a página** una sola vez. Mantener Automático. Mostrar el resultado y los indicadores.

DECIR.

Voy a solicitar otra vez la página cero. Antes de pulsar podemos anticipar el resultado porque la tabla dice que ya está en RAM, en el marco cero. Ahora aparece PAGE_HIT: un acierto de página.

No fue necesario traerla de secundaria ni encontrar otro espacio. La RAM sigue utilizando cuatro kilobytes y el número de marcos ocupados continúa en cuatro. La aplicación tampoco crea una segunda P0 por solicitarla de nuevo.

El contador de fallos permanece en cuatro. Tenemos cinco solicitudes completadas hasta este momento: cuatro encontraron una página ausente y la última encontró una presente. Esa diferencia es la que expresan Page Fault y Page Hit.

Un detalle importante es que utilizamos la página cero después de llenar la RAM. Eso la convierte en una página consultada recientemente, pero no cambia cuándo entró a la memoria. El orden de llegada continúa siendo P0, P1, P3 y P2.

Esto nos ayudará a identificar FIFO. Esa política observa el orden de carga: primero en entrar, primero en salir. No toma la decisión contando cuántas veces usamos una página ni eligiendo la que tiene el número más pequeño por definición. En este ejemplo P0 tiene el número más pequeño y también entró primero, pero la regla que importa es su antigüedad de carga.

Si preguntamos cuál saldrá al pedir otra página ausente, debemos mirar ese orden. Acabamos de usar P0, pero sigue siendo la más antigua. Deivid va a comprobarlo con la siguiente solicitud y después utilizará la tabla para calcular direcciones.

PAUSA. Preguntar: “Si pedimos P4 y no hay marcos libres, ¿cuál debería salir con FIFO?”. Esperar una respuesta y entregar el control sin solicitar P4 todavía.

HACER. Abrir **Inicio**, comprobar el mismo escenario y señalar **Page Faults: 4**. Volver a **Paginación**, seleccionar Chrome y dejar Automático. No solicitar otra página.

COMPROBAR. RAM **4 KB**, secundaria **2 KB**, Faults **4**, P0 en marco **0**. Pasar a Deivid a las **24:00**.

Deivid de 24 a 27 minutos

Provocar el reemplazo FIFO

HACER. Mostrar la parte FIFO de la diapositiva 7 y volver a **Paginación**. Mantener el escenario de la exposición y Chrome. Elegir **Página 4 · DISCO** y pulsar **Acceder a página** en Automático.

DECIR.

Soy Deivid Alberto Guerra Carpio. Vamos a comprobar la predicción. P4 está fuera de RAM y todos los marcos están ocupados. Para atender esta solicitud hace falta un Page Fault y también un reemplazo.

FIFO significa First In, First Out: primero en entrar, primero en salir. La página que cargamos antes que las demás fue P0. Por eso el sistema la devuelve al almacenamiento secundario y utiliza su marco para cargar P4.

En el resultado debemos observar que la víctima es Chrome P0. La tabla ahora muestra a P4 presente, en el marco cero; P0 vuelve a aparecer sin marco y con estado DISCO.

El último Hit de P0 no evitó su salida. Esto nos permite distinguir FIFO de una política que elige según el uso reciente. Aquí no estamos aplicando LRU; estamos respetando la fecha de entrada a RAM.

También podemos comprobar que el total utilizado sigue siendo de cuatro kilobytes. No se agregó un quinto marco. Una página salió y otra entró. La secundaria sigue utilizando dos kilobytes porque P4 dejó ese almacenamiento y P0 pasó a ocupar su lugar.

La página expulsada no se eliminó del proceso. Si más adelante la necesitamos, todavía podremos solicitarla y traerla otra vez. La ubicación es temporal y por eso debemos consultar siempre la tabla actual, no memorizar un marco como si fuera permanente.

Hasta ahora acumulamos cinco fallos y un Hit en seis solicitudes. Con el mapa actualizado vamos a ver cómo esa ubicación se utiliza para encontrar una posición concreta dentro de una página.

COMPROBAR. PAGE_FAULT, marco **0**, víctima **Chrome P0**. Mapa: **P4, P1, P3, P2** en marcos **0, 1, 2, 3**. En disco: **P0 y P5**. Faults **5**.

Deivid de 27 a 30 minutos

Traducir la dirección lógica 3500

HACER. Mostrar diapositiva 8. Decir la aclaración inicial. Abrir **Traducción de direcciones**, comprobar **Escenario** y elegir **Proceso: Chrome**. En **Dirección en bytes, desde cero**, escribir **3500** y pulsar **Traducir dirección**.

DECIR.

La lámina utiliza una tabla ilustrativa donde P1 está en el marco cinco. Ahora resolveremos otro ejemplo con la tabla actual de nuestra aplicación. En nuestro escenario solo existen los marcos cero al tres.

El número tres mil quinientos es una dirección lógica: la posición del byte que Chrome quiere consultar dentro de su propia memoria. No estamos agregando tres mil quinientos bytes a la RAM y tampoco estamos indicando el tamaño del proceso.

Cada página tiene mil veinticuatro posiciones. P0 abarca de cero a mil veintitrés. P1 comienza en mil veinticuatro y P2 en dos mil cuarenta y ocho. P3 empieza en tres mil setenta y dos. Entonces la posición tres mil quinientos pertenece a P3.

Si restamos tres mil setenta y dos a tres mil quinientos obtenemos cuatrocientos veintiocho. Ese número indica cuánto avanzamos desde el comienzo de P3. A ese avance dentro de la página lo llamamos offset o desplazamiento.

Ahora consultamos la tabla. P3 está alojada en el marco dos. Como cada marco también tiene mil veinticuatro bytes, el marco dos comienza en la dirección física dos mil cuarenta y ocho. Conservamos el avance de cuatrocientos veintiocho y obtenemos dos mil cuatrocientos setenta y seis.

La dirección lógica fue 3500 y la dirección física simulada es 2476. Son dos formas de localizar el mismo dato: una desde el espacio del proceso y otra desde su ubicación actual en RAM. La tabla establece la relación entre ambas.

COMPROBAR. Página 3, **offset 428**, **P3 → marco 2**, dirección física **2476 bytes**.

HACER. Señalar o escribir **3500 = 3 × 1024 + 428** y **2476 = 2 × 1024 + 428**. Señalar que el desplazamiento **428** se conserva.

Deivid de 30 a 33 minutos

Consultar una página que no está en RAM

HACER. Sin cambiar de escenario ni proceso, escribir **100** en **Dirección en bytes, desde cero** y pulsar **Traducir dirección**. Mostrar el aviso de página ausente. Si hace falta, regresar a **Paginación**, seleccionar Chrome y señalar P0 en la tabla, sin solicitarla todavía.

DECIR.

Ahora vamos a consultar la dirección lógica cien. Elegimos cien porque pertenece a la página cero: todavía no alcanzamos el tamaño de una página completa. El desplazamiento es cien bytes desde el inicio de P0.

Sin embargo, P0 fue precisamente la página que acabamos de sacar con FIFO. La cuenta puede identificar la página y su desplazamiento, pero la tabla nos indica que esa página no tiene un marco actual. Por eso no aparece una dirección física disponible.

Esto no significa que cien sea una dirección incorrecta del proceso. Chrome tiene seis kilobytes, por lo que esa posición sí pertenece a su espacio lógico. El problema es la presencia: la página que la contiene está en almacenamiento secundario.

También debemos distinguir la consulta de traducción de una solicitud de CPU. Esta pantalla consulta la tabla y realiza el cálculo. No carga por sí sola la página ni registra otro acceso de paginación. Si queremos volver a tener P0 en RAM, debemos solicitarla desde Paginación.

Por eso el contador de fallos continúa en cinco. Cambiar el número en la calculadora no produce automáticamente otro Page Fault guardado. El resultado nos está explicando qué condición falta para poder localizar físicamente el dato.

El siguiente paso será regresar a Paginación y pedir esa página. Como la RAM sigue llena, habrá otro reemplazo. La página que ahora lleva más tiempo cargada es P1, porque P0 ya había salido. Así podremos observar que P0 puede regresar a una ubicación diferente de la que tenía al principio.

PAUSA. Señalar las palabras **sin marco**. Preguntar: “¿Debemos inventar una dirección física o cargar primero la página?”. Confirmar: “Primero necesitamos que tenga una ubicación en RAM”.

COMPROBAR. Página **0**, offset **100**, **sin marco**, sin dirección física. Faults siguen en **5**.

Deivid de 33 a 36 minutos

Recargar P0 y comprobar que cambió su ubicación

HACER. Abrir **Paginación**, comprobar el mismo escenario y seleccionar **Chrome**. En Automático solicitar **Página 0 · DISCO**. Después abrir **Traducción de direcciones**, elegir escenario y Chrome, escribir **100** y pulsar **Traducir dirección**.

DECIR.

Solicitamos nuevamente P0. Está ausente y no tenemos un marco libre, así que ocurre otro fallo con reemplazo. FIFO elige P1, que es la más antigua de las páginas que permanecían cargadas.

P1 estaba en el marco uno. Ese espacio queda disponible y lo ocupa P0. La página cero vuelve a RAM, pero esta vez en el marco uno. No es obligatorio que regrese al marco cero; ese marco ahora contiene P4.

Regresamos a la traducción y escribimos exactamente la misma dirección lógica: cien. La página sigue siendo P0 y el desplazamiento sigue siendo cien. Lo que cambió es el número de marco. La tabla ahora indica uno.

El marco uno comienza en mil veinticuatro. Si avanzamos cien bytes llegamos a mil ciento veinticuatro. Por eso ahora tenemos una dirección física de 1124. La posición que pide el proceso no cambió: cambió el lugar donde se aloja su página en RAM.

Esta demostración relaciona los tres elementos: tabla de páginas, política de reemplazo y traducción. El proceso utiliza direcciones de su espacio lógico; el sistema consulta dónde están las páginas y puede moverlas conservando esa organización lógica.

Nuestro resultado final es cuatro marcos ocupados: P4 en el cero, P0 en el uno, P3 en el dos y P2 en el tres. P1 y P5 están fuera de RAM. Completamos siete solicitudes: seis fallos y un Hit. Las consultas de traducción no aumentaron esos contadores.

Con paginación usamos unidades del mismo tamaño. Rebeca va a mostrar cómo se representan regiones de distintos tamaños y por qué debemos validar sus límites antes de acceder a ellas.

COMPROBAR. Segundo reemplazo: sale **P1**, entra **P0** en marco **1**. Traducción: **1 × 1024 + 100 = 1124**. Faults **6**. Entregar el control a Rebeca a las **36:00**.

HACER antes de entregar. Abrir **Inicio**, comprobar el escenario de paginación y mostrar **Page Faults: 6**. Así se verifica en pantalla el conteo que llevamos en la hoja.

Rebeca de 36 a 39 minutos

Introducir segmentos y crear el segundo escenario

HACER. Mostrar diapositiva 9. Abrir **Segmentación**. En **Nuevo escenario**, ingresar **Nombre: Exposición G3 Segmentación 01** y **RAM total (KB): 16**; pulsar **Crear escenario**. Anotar su **#ID**. En **Crear proceso**, ingresar **Nombre: Editor** y **Tamaño del proceso (KB): 4**; pulsar **Crear proceso**.

DECIR.

Mi nombre es Rebeca Alizon Najarro Duarte. En paginación vimos partes iguales. En segmentación organizamos el proceso en regiones con una función, por ejemplo Código, Datos, Stack y Heap. Sus tamaños pueden ser diferentes.

Código representa las instrucciones del programa. Datos representa información que utiliza. Stack, o pila, se relaciona con llamadas a funciones y variables temporales. Heap representa la memoria que el programa solicita dinámicamente. Aquí utilizamos esas partes como un ejemplo educativo; el tamaño de cada una lo elegiremos manualmente.

Cada segmento tiene una base y un tamaño o límite. La base indica en qué dirección de RAM comienza. El tamaño indica cuántos bytes pertenecen a esa región. Para acceder utilizaremos un desplazamiento desde la base y primero comprobaremos si se mantiene dentro del segmento.

Creamos otro escenario, esta vez de segmentación, con dieciséis kilobytes de RAM. Son dieciséis mil trescientos ochenta y cuatro bytes. Esta pantalla no solicita tamaño de página porque no vamos a dividir este escenario en marcos.

Ahora creamos Editor con un tamaño declarado de cuatro kilobytes, equivalente a cuatro mil noventa y seis bytes. Crear el proceso todavía no ocupa regiones de RAM. La ocupación aparecerá cuando agreguemos sus segmentos activos.

La organización permitirá que las partes del proceso estén separadas. Sin embargo, cada segmento individual ocupará una región continua. Vamos a verlo al elegir sus bases con espacios entre ellas.

COMPROBAR. El escenario de segmentación y el proceso Editor quedan seleccionados. Aún no hay segmentos activos. No agregar campos de secundaria ni página: ese formulario no los tiene.

Rebeca de 39 a 42 minutos

Crear los segmentos y leer su tabla

HACER. En **Agregar segmento**, completar **Nombre**, **Base (bytes)** y **Tamaño / límite (bytes)**. Pulsar **Agregar segmento** después de cada fila. Completar los campos mientras se explica cada región, con apoyo de otro integrante en el teclado si hace falta, para no sumar toda la lectura y toda la captura como actividades separadas. No modificar el escenario ni el proceso.

Nombre	Base bytes	Tamaño límite bytes
Código	1000	1200
Datos	4000	800
Stack	7000	600
Heap	9000	1000

DECIR.

Agregamos Código con base mil y tamaño mil doscientos bytes. Su número se asigna automáticamente y será el segmento cero. La región empieza en mil y termina en dos mil ciento noventa y nueve, porque contamos también la primera posición.

Después agregamos Datos en la base cuatro mil con tamaño ochocientos. Stack empieza en siete mil y mide seiscientos. Heap empieza en nueve mil y mide mil bytes. Los tamaños son diferentes, pero todos están dentro de la RAM de este escenario.

En la tabla podemos leer el número, el nombre, la base, el tamaño y el estado. Estas columnas reemplazan la relación de páginas y marcos que observamos antes: ahora cada entrada describe directamente una región y sus límites.

Al bajar al mapa vemos Código de mil a dos mil ciento noventa y nueve; Datos de cuatro mil a cuatro mil setecientos noventa y nueve; Stack de siete mil a siete mil quinientos noventa y nueve; y Heap de nueve mil a nueve mil novecientos noventa y nueve.

Cada segmento forma una región continua. Las cuatro regiones, consideradas como partes del mismo proceso, no están pegadas entre sí. Entre ellas aparecen espacios libres que también forman parte del mapa. Esto nos ayuda a diferenciar una región continua de un proceso completo que puede estar distribuido.

Los cuatro tamaños suman tres mil seiscientos bytes. Esa es la RAM utilizada que debemos observar. La aplicación no presenta el resto del tamaño declarado de Editor como un bloque reservado; por eso no lo vamos a llamar fragmentación interna. La ocupación que estamos mostrando corresponde a estos cuatro segmentos activos.

COMPROBAR. Código **S0**, Datos **S1**, Stack **S2**, Heap **S3**. RAM total **16384 bytes**, utilizada **3600**, disponible **12784**. Señalar las bases y los espacios libres.

Rebeca de 42 a 45 minutos

Acceder dentro del segmento Código

HACER. Bajar a **Direccionamiento por segmentación**. En **Segmento**, elegir **0 · Código · límite 1200**. Escribir **100** en **Offset en bytes** y pulsar **Acceder**. Después sustituir el offset por **1199** y pulsar **Acceder** otra vez.

DECIR.

Vamos a utilizar Código para realizar un acceso. Su base es mil y su tamaño es mil doscientos bytes. La dirección que nos interesa se expresa como un segmento y un desplazamiento dentro de él.

Primero elegimos el segmento Código y escribimos un offset de cien. Offset significa cuánto avanzamos desde el inicio de la región. Antes de sumar, debemos revisar que ese avance sea menor que el tamaño del segmento.

Cien es menor que mil doscientos, así que está permitido. Sumamos la base mil más el desplazamiento cien y obtenemos la dirección física mil cien. La aplicación muestra SEGMENT_ACCESS, es decir, un acceso válido al segmento.

Ahora probaremos el borde del segmento. Cambiamos el desplazamiento a mil ciento noventa y nueve. También es menor que mil doscientos, por lo que sigue siendo válido. La dirección final es mil más mil ciento noventa y nueve, igual a dos mil ciento noventa y nueve.

Ese es el último byte de Código que vimos en el mapa. La tabla, el intervalo y el cálculo están describiendo la misma región. Si el tamaño es mil doscientos, tenemos mil doscientas posiciones, pero sus desplazamientos comienzan en cero. El último desplazamiento es tamaño menos uno.

No debemos confundir la base con un desplazamiento. La base ubica la región en RAM; el desplazamiento ubica el dato dentro de ella. También debemos distinguir el tamaño del número de la última posición: mil doscientos bytes no significa que se permita un offset de mil doscientos.

Vamos a comprobar esa diferencia con el siguiente acceso, que saldrá del límite por exactamente una posición.

COMPROBAR. Offset **100** → **dirección 1100**; offset **1199** → **dirección 2199**. Ambos con **SEGMENT_ACCESS**. Los segmentos siguen ocupando **3600 bytes**; acceder no agranda sus regiones.

Rebeca de 45 a 48 minutos

Rechazar un acceso fuera del límite

HACER. Mantener seleccionado Código. Ingresar **1200** en **Offset en bytes** y pulsar **Acceder**. Mostrar el resultado rojo y señalar el límite **1200** y el último offset válido **1199**.

DECIR.

Ahora el offset es mil doscientos. La comparación ya no se cumple, porque mil doscientos no es menor que mil doscientos. Hemos intentado acceder justo después del último byte permitido.

Aparece SEGMENTATION_FAULT y no se devuelve una dirección física válida. La protección consiste en revisar el límite antes de permitir el acceso. No debemos sumar la base y presentar dos mil doscientos como si fuera una dirección autorizada de Código.

Aunque el mapa muestre memoria libre después de Código, esa memoria no pertenece a este segmento. Tener espacio libre en RAM no concede permiso para que el proceso lo utilice mediante un desplazamiento fuera de su región.

Este resultado es distinto del Page Fault que mostramos antes. En un Page Fault, la página puede pertenecer al proceso y necesitar una carga a RAM. Aquí el desplazamiento excede los límites del segmento seleccionado. El simulador rechaza el acceso y registra lo ocurrido; no está mostrando que Editor se haya cerrado automáticamente.

También existe una validación al crear segmentos: sus regiones no pueden superponerse. Una región no debe invadir posiciones que ya ocupa otra región activa. El límite protege el acceso y la validación de intervalos protege la creación.

Con esto mostramos una tabla de segmentos, el significado de base y tamaño y dos accesos permitidos frente a uno rechazado. Carlos comparará esta organización con la paginación y con la asignación de un proceso completo en una región continua.

HACER si quedan 40 segundos. En **Agregar segmento**, escribir **Nombre: Prueba de solapamiento**, **Base: 1500**, **Tamaño / límite: 256** y pulsar **Agregar segmento**. Mostrar el rechazo porque invade Código. La tabla debe conservar cuatro segmentos y **3600 bytes** usados. Este ejemplo es opcional; si van atrasados, pasar directamente a Carlos.

COMPROBAR. El acceso con offset **1200** se rechaza. No afirmar que se eliminaron segmentos o se finalizó el proceso. Entregar el control a las **48:00**.

Carlos de 48 a 51 minutos

Comparar una solicitud con los huecos de memoria

HACER. Mostrar diapositiva 10. Abrir **Comparación de asignación**. En **Tamaño solicitado en bytes**, ingresar **8192** y pulsar **Comparar asignación**. Señalar los huecos de **3 KB** y **6 KB** y las tres columnas.

DECIR.

Soy Carlos Lopez Urizar. Ahora compararemos las tres técnicas con las mismas condiciones iniciales. Esta pantalla tiene un caso calculado independiente de los escenarios que acabamos de crear. Cada comparación parte de la misma distribución y no altera aquellos datos.

La lámina muestra una solicitud de siete kilobytes. En vivo probaremos ocho kilobytes para ver el mismo problema con otra cantidad. La RAM del comparador tiene dieciséis kilobytes. Los procesos A y B ocupan siete y quedan nueve libres, repartidos en dos huecos: uno de tres y otro de seis.

Escribimos ocho mil ciento noventa y dos bytes, que equivalen a ocho kilobytes. Hay nueve kilobytes libres en total, pero el mayor espacio continuo tiene seis. Por eso la asignación contigua rechaza la solicitud: necesita colocar el proceso completo en un solo intervalo y no encuentra uno suficiente.

Este problema es fragmentación externa. La memoria libre está repartida en huecos separados. Sumar esos espacios no los convierte automáticamente en una región continua de ocho kilobytes.

La paginación sí acepta el ejemplo. Divide la solicitud en ocho páginas de un kilobyte y puede distribuirlas entre marcos de los dos huecos. No necesita que todos los marcos del proceso estén juntos.

La segmentación también lo acepta con la división que utiliza este comparador: Código de cinco kilobytes y Datos de tres. Código cabe en el hueco de seis y Datos en el de tres. Cada segmento es continuo, aunque el proceso quede distribuido.

Esta aceptación no significa que cualquier conjunto de segmentos siempre encaje. Depende del tamaño de cada parte y de los huecos disponibles. Lo que comprobamos aquí es una distribución posible con estos números.

COMPROBAR. Solicitud **8192**: Contigua **Rechazada**, Paginación **Aceptada**, Segmentación **Aceptada**. Las dos técnicas aceptadas reservan **8192 bytes** y dejan **1024 libres**.

Carlos de 51 a 54 minutos

Explicar fragmentación interna y externa

HACER. Cambiar **Tamaño solicitado en bytes** a **7500** y pulsar **Comparar asignación**. Señalar **Reserva nueva**, **Fragmentación interna** y **Memoria libre final**. Si queda tiempo, probar **6144** para mostrar que Contigua también puede aceptar una solicitud que cabe en un hueco.

DECIR.

Ahora solicitamos siete mil quinientos bytes. Seguimos teniendo suficiente memoria libre total, pero la asignación contigua vuelve a fallar porque la solicitud supera el hueco más grande, de seis mil ciento cuarenta y cuatro bytes.

Con paginación necesitamos ocho páginas. Siete páginas solo darían siete mil ciento sesenta y ocho bytes, y todavía faltarían posiciones para completar la solicitud. Como los marcos tienen tamaño fijo, reservamos ocho, equivalentes a ocho mil ciento noventa y dos bytes.

La diferencia entre la reserva y lo solicitado es seiscientos noventa y dos bytes. Ese espacio sobra dentro del último marco asignado y no se entrega a otro proceso en este modelo. Esto es fragmentación interna: el sobrante está dentro de una unidad reservada.

La fragmentación externa era diferente: eran huecos libres entre regiones asignadas. Por eso no debemos usar los dos términos como si significaran lo mismo. En un caso el espacio libre está separado afuera; en el otro, el desperdicio queda dentro de una unidad que ya se reservó.

En este ejemplo la segmentación reserva exactamente los siete mil quinientos bytes que necesita. Eso permite evitar el relleno de una última página, pero mantiene la necesidad de encontrar un hueco continuo para cada segmento. No estamos afirmando que una técnica sea superior en todas las situaciones.

Si reducimos la solicitud a seis mil ciento cuarenta y cuatro bytes, cabe completa en el hueco de seis kilobytes y las tres técnicas aceptan. Esta prueba nos recuerda que la decisión depende de las condiciones: tamaño solicitado, tamaño de las unidades y distribución de los espacios libres.

COMPROBAR para 7500. Paginación: reserva **8192**, fragmentación interna **692**, libre final **1024**. Segmentación: reserva **7500**, fragmentación interna **0**, libre final **1716**. Contigua: **Rechazada**.

PAUSA. Escribir **8192 - 7500 = 692** y preguntar dónde se encuentra ese sobrante. Confirmar: “Dentro del último marco asignado”.

Carlos de 54 a 57 minutos

Explicar construcción verificación y uso de IA

HACER. Mostrar diapositiva 11. Si hay tiempo, abrir **Entregables académicos** y señalar **Manual de uso**, **Informe técnico**, **Presentación** y **Video**; no iniciar la reproducción del video de tres minutos. Regresar a la presentación.

DECIR.

Además de mostrar resultados, el proyecto debía conservar una explicación verificable de cómo se construyó. La aplicación guarda escenarios, procesos, asignaciones y eventos. Esto permite volver a consultar un estado y revisar las acciones realizadas.

La separación de roles ayuda a controlar quién modifica la simulación. Un observador puede estudiar los resultados; un operador puede ejecutar las acciones para las que tiene permiso; y un administrador prepara la configuración y realiza la gestión correspondiente. Las validaciones se comprueban en el servidor.

La diapositiva muestra evidencia de una revisión del ocho de octubre: se documentaron seiscientas cincuenta y cinco pruebas aprobadas, una omisión condicional y cuatro mil quinientas setenta y cinco aserciones. Estas cifras pertenecen a esa revisión histórica. No estamos diciendo que se acaban de ejecutar durante esta exposición.

También utilizamos herramientas de inteligencia artificial como apoyo para analizar, desarrollar, documentar y corregir el proyecto. Esa ayuda no sustituye comprobar lo que el sistema hace. Las instrucciones, decisiones y correcciones se conservaron en la documentación.

Durante la demostración acabamos de aplicar ese criterio de comprobación. No dimos por correcto un número solo porque aparecía en un texto: verificamos P3 en el marco dos, comprobamos el desplazamiento cuatrocientos veintiocho y validamos el límite mil doscientos con accesos permitidos y rechazados.

En Entregables académicos están el manual, el informe, la presentación y un video que ayudan a revisar el proyecto. Para esta exposición usamos además el recorrido que ensayamos y los nombres de nuestros propios escenarios. Con ese respaldo vamos a cerrar retomando los temas solicitados.

COMPROBAR. Presentar las cifras como históricas. El guion de una hora es un documento de apoyo aparte; no afirmar que existe un botón nuevo para descargarlo en la app.

Carlos de 57 a 60 minutos

Cerrar y responder preguntas dentro de la hora

HACER. Mostrar diapositiva 12. Dar el cierre hasta aproximadamente **58:40**. Reservar de **58:40 a 59:40** para una pregunta o dos respuestas muy breves; terminar a **60:00**.

DECIR para cerrar.

Para concluir, la tabla de páginas relaciona el espacio lógico del proceso con los marcos donde se encuentran sus páginas. Cuando una página está ausente, un acceso puede requerir cargarla y, si la RAM está llena, reemplazar otra. Con FIFO comprobamos que sale la que entró antes, aunque se haya utilizado recientemente.

En segmentación, la base ubica una región y el límite permite comprobar si el desplazamiento pertenece a ella. Por eso un offset de mil ciento noventa y nueve fue válido y uno de mil doscientos se rechazó para un segmento de mil doscientos bytes.

La comparación mostró que espacio libre total y espacio libre continuo son cantidades diferentes. También vimos la fragmentación interna de una página parcialmente utilizada. Las técnicas tienen ventajas y costos que dependen de cómo está organizada la memoria.

Estos resultados cumplen los temas centrales de nuestra tarea: tablas de páginas, fallos, tablas de segmentos, base y límite, asignación contigua y no contigua y conclusiones técnicas. Gracias por su atención. Podemos responder sus preguntas.

SI NO HAY PREGUNTAS, DECIR.

Para repasar, les planteamos dos preguntas. ¿La página cero siempre debe estar en el marco cero? No. Vimos que regresó al marco uno y que la misma dirección lógica cien se tradujo a 1124.

¿Tener nueve kilobytes libres garantiza poder ubicar ocho de forma contigua? No. Si el mayor hueco solo tiene seis, hace falta distribuir la solicitud o cambiar las condiciones de asignación.

SI HAY PREGUNTAS. Responder de forma breve. Enmer responde sobre objetivo y arquitectura; Lester sobre tabla, Hit y Fault; Deivid sobre FIFO y direcciones; Rebeca sobre segmentos; Carlos sobre comparación e IA. Las respuestas preparadas están en las páginas siguientes.

DECIR al finalizar.

Con esto terminamos la presentación de MemoryLab del Grupo 3. Muchas gracias, licenciado y compañeros.

Hoja rápida de resultados de paginación

Usar esta hoja para ensayar y para detectar un clic adicional. No hace falta leerla completa al público.

Acción en el escenario nuevo	Marco	RAM usada KB	Secundaria usada KB	Faults
Crear Chrome de 6 KB	Ninguno	0	6	0
Solicitar P0 paso a paso hasta el paso 8	0	1	5	1
Solicitar P1 en Automático	1	2	4	2
Solicitar P3 en Automático	2	3	3	3
Solicitar P2 en Automático	3	4	2	4
Solicitar P0 otra vez y obtener Hit	0	4	2	4
Solicitar P4 y expulsar P0	0	4	2	5
Consultar traducción de 3500	P3 en 2	4	2	5
Consultar traducción de 100 sin P0	Sin marco	4	2	5
Solicitar P0 y expulsar P1	1	4	2	6
Consultar traducción de 100 con P0	P0 en 1	4	2	6

Dos traducciones que deben poder explicar

3500. Las primeras tres páginas completan **3072 bytes**. Por eso la página es **3** y el desplazamiento es **3500 - 3072 = 428**. La tabla coloca P3 en el marco **2**. Ese marco empieza en **2048**, y **2048 + 428 = 2476**.

100 después de recargar P0. La página es **0** y el desplazamiento es **100**. P0 ahora está en el marco **1**. El marco empieza en **1024**, y **1024 + 100 = 1124**.

Estado final

Marco **0: P4**, marco **1: P0**, marco **2: P3**, marco **3: P2**. En secundaria: **P1 y P5**. Hay **7 solicitudes completadas**, **6 Faults**, **1 Hit** y **2 reemplazos FIFO**. Las traducciones no crean solicitudes de CPU ni alteran estos conteos.

Para repetir desde cero. Crear otro escenario nuevo con los mismos tamaños y un nombre distinto. Mantener el historial de los ensayos anteriores. No intentar recuperar el inicio pulsando más páginas ni reiniciando escenarios ajenos.

Hoja rápida de segmentación y comparación

Segmento	Base	Tamaño	Última dirección
Código S0	1000	1200	2199
Datos S1	4000	800	4799
Stack S2	7000	600	7599
Heap S3	9000	1000	9999

Total activo **3600 bytes**. RAM **16384 bytes**. Disponible **12784 bytes**. El tamaño declarado de Editor es **4096 bytes**; sus **496 bytes** restantes no se muestran como RAM reservada ni son la fragmentación interna del comparador.

Acceso a Código	Comparación	Resultado
Offset 100	100 < 1200	SEGMENT_ACCESS y dirección 1100
Offset 1199	1199 < 1200	SEGMENT_ACCESS y dirección 2199
Offset 1200	1200 no es menor que 1200	SEGMENTATION_FAULT sin dirección válida

Prueba opcional de solapamiento. Agregar base **1500**, tamaño **256**, nombre **Prueba de solapamiento**. Debe rechazarse por invadir Código. Quedan los mismos cuatro segmentos activos.

Comparador con huecos de 3 y 6 KB

Tiene **9216 bytes libres** en total y un hueco máximo de **6144**. Es un ejemplo independiente que no guarda cambios en los escenarios.

Solicitud bytes	Contigua	Paginación	Segmentación
8192	Rechazada	Aceptada reserva 8192	Aceptada reserva 8192
7500	Rechazada	Aceptada reserva 8192 y sobrante 692	Aceptada reserva 7500
6144 opcional	Aceptada	Aceptada	Aceptada

En **7500**, los **692 bytes** son fragmentación interna de la última página. Los huecos libres entre regiones ilustran fragmentación externa. La aceptación de segmentación depende de la partición concreta de Código y Datos que utiliza el comparador.

Preguntas que puede hacer el docente

Por qué el proceso tiene seis páginas

Porque declaramos un tamaño de 6 KB y cada página mide 1 KB. Se requieren seis páginas, numeradas de cero a cinco. Si el tamaño no fuera múltiplo exacto, se necesitaría una última página parcialmente utilizada.

Página y marco son lo mismo

Tienen igual tamaño en este esquema, pero la página es una parte del proceso y el marco es un espacio de RAM que puede alojarla. Por eso P3 pudo ocupar el marco 2.

RAM y caché son lo mismo

Son niveles distintos de la memoria de un computador. En esta app estudiamos páginas y marcos de RAM y almacenamiento secundario simulado; no estamos mostrando la caché del procesador. Que un Hit evite otra carga no convierte el mapa en una simulación de caché de CPU.

Page Fault significa que una dirección es inválida

En nuestro ejemplo la página pertenece al proceso, pero está ausente de RAM y debe cargarse. Una dirección fuera del espacio válido es un problema distinto. La validación de límites también se aplica en los formularios.

Por qué salió P0 si acabábamos de usarla

Porque FIFO decide por la fecha de carga. El Hit no cambia esa antigüedad. Una política como LRU toma en cuenta el uso reciente, pero esa no es la política mostrada aquí.

Por qué la dirección lógica 100 terminó en 1124

P0 pasó a estar en el marco 1. La posición cien del proceso sigue estando cien bytes dentro de P0; al ubicar ese desplazamiento en el marco 1 se calcula 1024 + 100.

Por qué 1200 no es válido si el tamaño es 1200

Porque hay 1200 desplazamientos numerados del 0 al 1199. La condición correcta es offset menor que tamaño, no menor o igual.

Un fallo de segmentación cerró Editor

La aplicación rechaza y registra ese acceso sin entregar una dirección física válida. En esta demostración no finaliza automáticamente el proceso ni elimina sus segmentos.

Más respuestas y manejo del tiempo

Por qué se rechazaron ocho KB con nueve libres

Porque asignación contigua exige ocho KB juntos y el hueco más grande tenía seis. La suma de los huecos era suficiente, pero su distribución no lo era.

La paginación siempre exige cargar todo el proceso en RAM

No. En nuestro escenario principal solo había cuatro de seis páginas residentes. El comparador utiliza otro caso: calcula si puede alojar la solicitud completa usando los huecos del ejemplo. Hay que distinguir ambos modelos de demostración.

La segmentación elimina todos los problemas de fragmentación

No. Permite segmentos de distintos tamaños, pero cada uno necesita una región continua y puede haber fragmentación externa. En el comparador se muestra una partición que sí cabe, no una garantía para cualquier partición.

Qué hizo la inteligencia artificial y qué comprobaron ustedes

La IA apoyó análisis, desarrollo y documentación. Para exponer, el grupo debe ensayar y comprender las reglas; durante la demostración comprobamos tablas, marcos, límites y cálculos. La documentación conserva evidencia de instrucciones y correcciones. No atribuir revisiones personales que no hayan realizado.

Qué consumió realmente la aplicación

Como cualquier aplicación web utiliza recursos normales para funcionar, pero las cantidades configuradas en sus diagramas pertenecen al modelo. No son una medición del consumo real de Chrome, Editor o del sistema operativo anfitrión.

Si el ritmo no coincide con el reloj

Si van dos minutos atrasados. Acortar arquitectura y modelo persistente a una frase cada uno. Omitir la prueba opcional de solapamiento, la solicitud opcional de 6144 y la visita a Entregables. Conservar tabla de páginas, Fault, tabla de segmentos, base/límite, comparación y conclusiones. No recuperar tiempo saltándose el paso 6 de la carga.

Si van adelantados. Pedir al público que prediga la víctima de FIFO antes del clic, que encuentre P3 en el mapa o que explique por qué 1200 excede el límite. Darles tiempo para responder y después mostrar el resultado. Repetir ejemplos explicando lo que cambia, sin hacer accesos extra que alteren el orden de carga.

Si aparece una pregunta larga. Decir: “Para respetar el tiempo vamos a dar una respuesta breve y podemos ampliar ese punto al terminar”. Reservar la diapositiva 12 y el cierre para el último tramo.

Qué hacer si algo no coincide durante la demostración

Hay números diferentes en la tabla. Revisar primero nombre y #ID del escenario, luego proceso y modo de simulación. El caso debe ser nuevo y manual. Comprobar que las primeras cargas fueron P0, P1, P3 y P2. No solicitar páginas al azar para intentar forzar el resultado.

P0 ya estaba en RAM al empezar. El escenario ya fue utilizado. Antes de la exposición preparar otro con un nombre nuevo. Si ocurre en vivo, explicar el estado real que muestra la tabla y recuperar el recorrido con un escenario nuevo; recortar las actividades opcionales para compensar. No llamar Fault a un Hit que esté en pantalla.

No aparece un botón para guardar o crear. Revisar que la cuenta tiene rol Administrador y que el escenario elegido corresponde al módulo. Configuración de memoria corresponde a paginación; el escenario de segmentación se crea desde Segmentación.

Se detuvo el paso a paso. Mantener la misma página, escenario y proceso y seguir con Siguiente hasta el paso 8. Cerrar recorrido solo limpia la explicación local: no deshace una carga ya confirmada. El cambio efectivo del Fault ocurre al llegar al paso 6.

La traducción muestra sin marco. Revisar si esa página fue expulsada. Volver a Paginación y solicitarla solo cuando el guion lo indique. Después regresar a Traducción y comprobar la tabla actual. La calculadora no realiza la carga automáticamente.

El comparador no da los valores esperados. Escribir 8192 o 7500 en bytes, no 8 ni 7,5. La coma decimal y el punto de miles no se deben ingresar en este campo entero.

No responde la aplicación. Usar la presentación y el manual ya descargados para explicar las reglas. Decir: “En este momento mostraremos la evidencia guardada y explicaremos qué resultado debe producir el recorrido”. Identificar esas imágenes como evidencia previa; no afirmar que son operaciones ejecutadas en vivo.

Comprobación final del ensayo

Cada integrante debe poder explicar su tabla sin leer solo las cifras. Ensayar los cambios entre presentación y app, respetar el orden de cargas y medir los cinco turnos. Registrar cuánto tardan realmente y ajustar las pausas antes del día de la exposición.

Esta guía se elaboró con la solicitud académica, la presentación existente, el manual y la inspección de vistas, servicios y reglas de MemoryLab. No crea escenarios ni modifica datos al abrirla. Su recorrido fue revisado contra el código; el grupo debe practicar los clics en escenarios nuevos antes de exponer.

Fuentes del proyecto: **docs/evidencias/solicitud-inicial.md**, **docs/manual-de-uso.md**, **docs/uso-ia.md**, **config/memorylab.php**, **resources/views/livewire/**, **app/Services/** y **output/presentations/memorylab-presentacion.pptx**.